

ME403 — Lab 2: Jacobian and Inverse Kinematics

Sabanci University

Complete the following tasks on a robot of your choice (Dobot Magician or MG400). Start in simulation; move to hardware only after simulation results are stable.

Assignment

Task 1 — Jacobian. Build the 4×4 Jacobian matrix of the end-effector position with respect to joint angles. Your Jacobian function should accept the current joint configuration $\mathbf{q} \in \mathbb{R}^4$ and return $J(\mathbf{q}) \in \mathbb{R}^{3 \times 4}$ (position rows only — ignore orientation for this lab).

Task 2 — Finite-difference validation. Verify your analytic Jacobian against a numerical finite-difference approximation. For each joint i , perturb by $\delta = 0.01$ rad and estimate the partial derivative:

$$\frac{\partial p}{\partial q_i} \approx \frac{f(\mathbf{q} + \delta \mathbf{e}_i) - f(\mathbf{q} - \delta \mathbf{e}_i)}{2\delta}$$

Report the mean squared error (MSE) between analytic and numerical columns over at least 10 random configurations inside the workspace.

Task 3 — Gauss-Newton IK and square trajectory. Implement a Gauss-Newton iterative IK solver:

$$\mathbf{q}_{k+1} = \mathbf{q}_k + \alpha J^\dagger(\mathbf{q}_k) (\mathbf{p}_{\text{target}} - f(\mathbf{q}_k))$$

where $J^\dagger = J^T(JJ^T)^{-1}$ is the right pseudoinverse and α is a step size ($0 < \alpha \leq 1$). Use your IK solver to command the robot to trace a square (four corner points) in Cartesian space. Collect actual measured positions at each waypoint and plot the XY and XZ trajectory.

Theoretical Background

The Position Jacobian

Given the forward kinematics $f: \mathbb{R}^4 \rightarrow \mathbb{R}^3$, the Jacobian is:

$$J(\mathbf{q}) = \frac{\partial f}{\partial \mathbf{q}} = \begin{bmatrix} \frac{\partial x}{\partial q_1} & \cdots & \frac{\partial x}{\partial q_4} \\ \frac{\partial y}{\partial q_1} & \cdots & \frac{\partial y}{\partial q_4} \\ \frac{\partial z}{\partial q_1} & \cdots & \frac{\partial z}{\partial q_4} \end{bmatrix} \in \mathbb{R}^{3 \times 4}$$

For the Magician FK ($R = L_1 \sin q_2 + L_2 \cos q_3$):

$$\begin{aligned} \frac{\partial x}{\partial q_1} &= -R \sin q_1, & \frac{\partial x}{\partial q_2} &= L_1 \cos q_2 \cos q_1, & \frac{\partial x}{\partial q_3} &= -L_2 \sin q_3 \cos q_1, & \frac{\partial x}{\partial q_4} &= 0 \\ \frac{\partial y}{\partial q_1} &= R \cos q_1, & & & & & & \text{(and so on for } y, z) \end{aligned}$$

Tip: Analytic vs Numerical

Computing the Jacobian analytically (by hand differentiation) is more accurate and faster. The numerical finite-difference check is useful to catch sign errors or wrong link lengths in your analytic version.

Gauss-Newton Inverse Kinematics

Given target position $\mathbf{p}^* \in \mathbb{R}^3$, find $\mathbf{q}$ such that $f(\mathbf{q}) = \mathbf{p}^*$. The Gauss-Newton update minimises $\|f(\mathbf{q}) - \mathbf{p}^*\|^2$:

$$\mathbf{q}_{k+1} = \mathbf{q}_k + \alpha J(\mathbf{q}_k)^T [J(\mathbf{q}_k) J(\mathbf{q}_k)^T]^{-1} (\mathbf{p}^* - f(\mathbf{q}_k))$$

Implementation Hints

- Start from $\mathbf{q}_0 = \mathbf{q}_{\text{current}}$ to stay near the present configuration.
- Use $\alpha = 0.5\text{--}1.0$. Smaller α is more stable near singularities.
- Stop when $\|\mathbf{p}^* - f(\mathbf{q}_k)\| < \varepsilon$ (e.g. $\varepsilon = 0.5$ mm).
- Cap iteration count (e.g. 50 steps) to prevent infinite loops.
- Add a small diagonal regularisation λI to JJ^T when near singularities: $(JJ^T + \lambda I)^{-1}$.

Robot Kinematics Quick Reference

Dobot Magician

$L_1 = 135$ mm, $L_2 = 147$ mm. J2 zero = upper arm vertical; J3 zero = forearm horizontal (absolute world angle).

$$R = L_1 \sin q_2 + L_2 \cos q_3, \quad z = L_1 \cos q_2 - L_2 \sin q_3, \quad x = R \cos q_1, \quad y = R \sin q_1$$

Home at $(q_1, q_2, q_3) = (0, 0, 0)$: TCP = (147, 0, 135) mm (bare flange, DOBOT_EE=none).

TCP Offset: Keep Sim and Real in Sync

The Jacobian is computed at the TCP. If your simulation uses DOBOT_EE=none (bare flange) but the real robot has a motor-flange offset, your IK targets will be 60 mm off in X.

Check and set the real-robot offset from Python:

```
import utils as U
print(U.get_tool_offset(bot)) # read
    current
U.set_tool_offset(bot, *U.MAGICIAN_TOOL_OFFSETS["none"]) # set bare
    flange
```

DOBOT MG400

$Z_{\text{base}} = 116$ mm, $L_1 = L_2 = 175$ mm. $J_3^{\text{fw}} = J_2 + J_3^{\text{body}}$ (handled by `utils.py`).

$$R = L_1 \cos q_2 + L_2 \cos(q_3^{\text{fw}} - q_2), \quad z = Z_{\text{base}} + L_1 \sin q_2 + L_2 \sin(q_3^{\text{fw}} - q_2)$$

Safe Z: 5–140 mm (Z cannot go negative). No TCP offset needed.

Due Date & Deliverables

Due Date: As announced by the instructor.

Deliverables: Complete these steps by the time of your lab session. A brief written report will be requested after the session.

Provided Files

<code>utils.py</code>	Robot connection, motion helpers, FK constants.
<code>interface.py</code>	Terminal interface to jog the robot and run your code.
<code>myCode.py</code>	Write your code here (inside <code>run()</code>).
<code>requirements.txt</code>	Python dependencies.

Quick Start — Simulation

Run in Simulation (No Hardware Needed)

```
# Magician (default)
python interface.py

# MG400
DOBOT_ROBOT_TYPE=mg400 python interface.py

# PyBullet backend
DOBOT_SIM_BACKEND=pybullet python interface.py
```

Run on Real Robot

```
# Magician (USB serial)
DOBOT_SIMULATION=0 DOBOT_ROBOT_TYPE=magician python interface.py

# MG400 (Ethernet -- set PC IP to 192.168.2.100 first)
DOBOT_SIMULATION=0 DOBOT_ROBOT_TYPE=mg400 DOBOT_MG400_ROBOT=1
python interface.py
```

At the interface prompt:

- `move q1 q2 q3 q4` — jog to joint angles (degrees)

- `x` — run `run()` in `myCode.py`
- `q` — quit safely

Setup Reference (if You Did Not Attend Earlier Labs)

Avoid PDF Copy-Paste for Commands

Type commands manually or copy from the digital version of this PDF. Some viewers insert extra spaces in paths (e.g. `. venv` instead of `.venv`).

Virtual Environment

Linux / macOS

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

Windows (PowerShell)

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
```

Dobot Magician (USB/Serial)

- Close DobotStudio before running Python (single-owner rule).
- Home once in DobotStudio (v1.9.4) after each power-on.
- Linux: `sudo usermod -a -G dialout $USER` then re-login.
- Set TCP offset via Python: `U.set_tool_offset(bot, *U.MAGICIAN_TOOL_OFFSETS["none"])`.

DOBOT MG400 (Ethernet)

Set PC Ethernet to static IP 192.168.2.100/24, then ping your robot:

```
ping 192.168.2.7      # Robot 1 (change to your assigned robot's IP)
```

Robot IPs: Robot 1 → 192.168.2.7, Robot 2 → 192.168.2.10, Robot 3 → 192.168.2.9, Robot 4 → 192.168.2.6.

SDK (not on PyPI — clone once):

```
git clone https://github.com/Dobot-Arm/TCP-IP-4Axis-Python.git \
  vendor/TCP-IP-4Axis-Python
```